Department of Computer and Software Engineering
SE: Machine Learning

Course Instructor:	**Date:**
Day:	**Semester:**

Lab 7: Agglomerative Clustering and Dendrograms

Lab Title	**CLO**	**BT**	**PLO**	**Weightage**
Agglomerative Clustering and Dendrograms				

Name	**Roll Number**	**Report /10**	**Viva /5**	**Total /15**
Muhammad Abdul Qadir	BSSE23105			

Checked on: _____

Signature: _____

1 Learning Outcomes

After completing this lab, students will be able to:

- Understand hierarchical clustering.
- Implement agglomerative clustering from scratch.
- Compare different linkage methods.
- Interpret dendrograms.

2 Dataset

We will use a synthetic dataset generated using:

`sklearn.datasets.make_blobs`

Dataset properties:

- Number of samples: 100
- Number of clusters: 3
- Features: 2 (for visualization)

3 Agglomerative Clustering

Agglomerative clustering is a bottom-up hierarchical clustering method.
Algorithm:

1. Start with each point as its own cluster.
2. Find the closest pair of clusters.
3. Merge them into a single cluster.
4. Repeat until desired number of clusters is reached.

4 Linkage Methods

Different linkage methods define cluster distance differently.

4.1 Single Linkage

$$d(C_1, C_2) = \min ||x - y||$$

Produces chain-like clusters.

4.2 Complete Linkage

$$d(C_1, C_2) = \max ||x - y||$$

Produces compact clusters.

4.3 Average Linkage

$$d(C_1, C_2) = \text{mean}||x - y||$$

Balanced clustering approach.

5 Part A: Distance Matrix (3 Marks)

Implement:

`compute_distance_matrix(X)`

Requirements:

- Compute pairwise Euclidean distances
- Output should be an NxN matrix
- Matrix must be symmetric
- Diagonal values must be zero

Solution

```
def compute_distance_matrix(X):
    number_of_datapoints = len(X)
    distance_matrix_result = np.zeros(
        (number_of_datapoints, number_of_datapoints))
    for row_index in range(number_of_datapoints):
        for column_index in range(number_of_datapoints):
            first_point = X[row_index]
            second_point = X[column_index]
            difference_vector =first_point - second_point
            euclidean_distance_val = np.sqrt(
                np.sum(difference_vector **2))
            distance_matrix_result[row_index][column_index] =
               euclidean_distance_val
    return distance_matrix_result
```

6 Part B: Linkage Distance (4 Marks)

Implement:

`linkage_distance(cluster1, cluster2, X, method)`

Support the following methods:

- Single linkage
- Complete linkage
- Average linkage

Solution

```
def linkage_distance(cluster1, cluster2, X, method="single"):
    all_distances_between_clusters = []
    for point_index_cluster1 in cluster1:
        for point_index_cluster2 in cluster2:
            pointA = X[point_index_cluster1]
            pointB =X[point_index_cluster2]
            distance_between_points = np.sqrt(
                np.sum((pointA- pointB)**2))
            all_distances_between_clusters.append(
                distance_between_points)
    if method =="single":
        final_linkage_distance = np.min(
            all_distances_between_clusters)
    elif method=="complete":
        final_linkage_distance =np.max(
            all_distances_between_clusters)
    elif method == "average":
        final_linkage_distance =np.mean(
            all_distances_between_clusters)
    else:
        raise ValueError(f"Unknown linkage method: {method}")
    return final_linkage_distance
```

7 Part C: Find Closest Clusters (3 Marks)

Implement:

`find_closest_clusters(clusters, X, method)`

Steps:

- Compute distance between all cluster pairs
- Return indices of closest clusters
- Return the corresponding distance

Solution

```
def find_closest_clusters(clusters, X, method):
    minimum_distance_found = np.inf
    closest_cluster_i = None
    closest_cluster_j =None
    total_number_of_clusters = len(clusters)
    for index_i in range(total_number_of_clusters):
        for index_j in range(index_i +1,
           total_number_of_clusters):
            current_cluster_one = clusters[index_i]
            current_cluster_two =clusters[index_j]
            computed_distance = linkage_distance(
                current_cluster_one, current_cluster_two,
                X, method)
            if computed_distance <minimum_distance_found:
                minimum_distance_found= computed_distance
                closest_cluster_i =index_i
                closest_cluster_j = index_j
    return closest_cluster_i, closest_cluster_j,
       minimum_distance_found
```

8 Part D: Agglomerative Clustering (5 Marks)

Implement:

`fit(X)`

Steps:

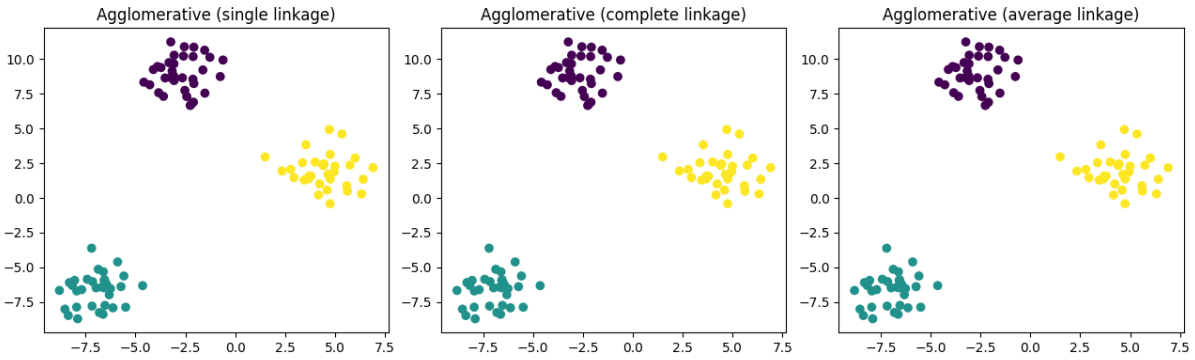
1. Initialize each point as its own cluster
2. Iteratively merge closest clusters
3. Stop when desired number of clusters is reached

Store:

- Cluster labels
- Merge history (for dendrogram)

Solution

```
def fit(self, X):
    total_samples = len(X)
    all_clusters = [[i] for i in range(total_samples)]
    while len(all_clusters) > self.n_clusters:
        idx_i, idx_j, merge_distance = find_closest_clusters(
            all_clusters, X, self.linkage)
        merged_new_cluster = all_clusters[idx_i] +all_clusters[
           idx_j]
        all_clusters = [c for cluster_idx, c in enumerate(
           all_clusters)
            if cluster_idx!=idx_i and cluster_idx !=idx_j]
        all_clusters.append(merged_new_cluster)
        self.history_.append((idx_i, idx_j, merge_distance))
    label_array = np.zeros(total_samples, dtype=int)
    for cluster_label, points_in_this_cluster in enumerate(
       all_clusters):
        for single_point_index in points_in_this_cluster:
            label_array[single_point_index]= cluster_label
    self.labels_ =label_array
```



9 Part E: Dendrogram (Bonus)

Implement:

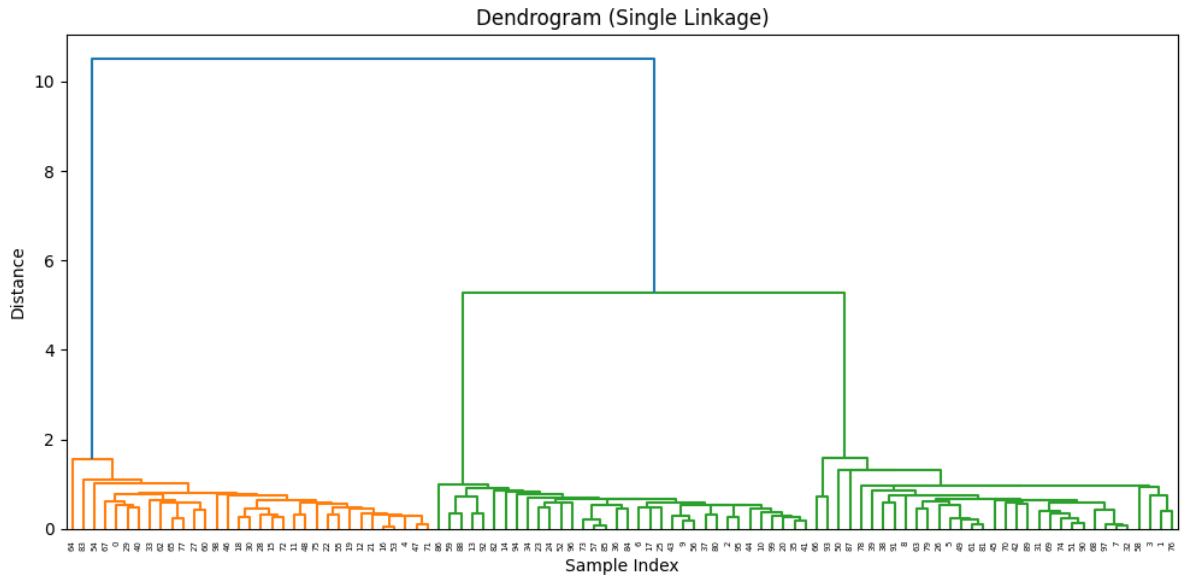
`plot_dendrogram(history)`

Steps:

- Convert merge history to linkage format
- Plot dendrogram using scipy

Solution

```
def plot_dendrogram(X):
    import scipy.cluster.hierarchy as sch
    linkage_computation_result = sch.linkage(X, method="single"
       )
    plt.figure(figsize=(10, 5))
    sch.dendrogram(linkage_computation_result)
    plt.title("Dendrogram (Single Linkage)")
    plt.xlabel("Sample Index")
    plt.ylabel("Distance")
    plt.tight_layout()
    plt.show()
```



10 Part F: Comparison with sklearn (Bonus)

Use:

`sklearn.cluster.AgglomerativeClustering`

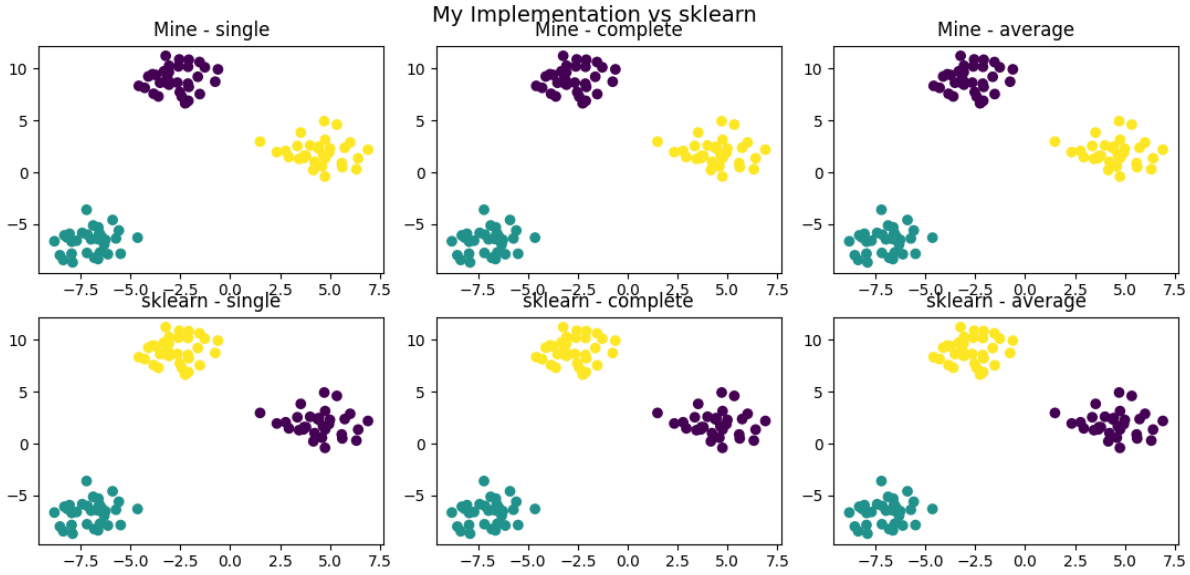
Compare:

- Cluster assignments
- Effect of linkage methods

Solution

```
def compare_with_sklearn(X):
    from sklearn.cluster import AgglomerativeClustering
    linkage_methods_to_test = ["single", "complete","average"]
    fig, axes_grid = plt.subplots(2, 3, figsize=(15, 8))
    for column_idx, current_method_name in enumerate(
            linkage_methods_to_test):
        my_clustering_model = MyAgglomerative(
            n_clusters=3, linkage=current_method_name)
        my_clustering_model.fit(X)
        my_predicted_labels =my_clustering_model.predict(X)
        axes_grid[0][column_idx].scatter(
            X[:,0], X[:,1], c=my_predicted_labels, cmap='
               viridis')
        axes_grid[0][column_idx].set_title(
```

```
            f"Mine - {current_method_name}")
        sklearn_model = AgglomerativeClustering(
            n_clusters=3, linkage=current_method_name)
        sklearn_predicted_labels = sklearn_model.fit_predict(X)
        axes_grid[1][column_idx].scatter(
            X[:,0], X[:,1], c=sklearn_predicted_labels, cmap='
               viridis')
        axes_grid[1][column_idx].set_title(
            f"sklearn - {current_method_name}")
    plt.suptitle("My Implementation vs sklearn", fontsize=14)
    plt.tight_layout()
    plt.show()
```



11 Expected Output

Your program should:

- Perform agglomerative clustering
- Display cluster plots
- Show differences between linkage methods
- (Bonus) Display dendrogram